PROYECTO DE GRADO · UTS

Mail Guard

Detección de phishing explicable para Gmail y Outlook



[Nombre de la estudiante] · [Programa] · [Fecha de sustentación]

EL PROBLEMA

Un correo de phishing solo necesita un clic

Suplanta a quien conoces

Bancos, billeteras digitales, la DIAN o tu propia universidad.

Presiona para actuar rápido

«Tu cuenta será bloqueada en 24 horas».

Esconde el destino real

Enlaces acortados, direcciones IP y dominios casi idénticos.

Cuando pasa el filtro, el usuario queda solo frente a una pregunta: **¿puedo confiar en esto?**

LA SOLUCIÓN

Mail Guard responde justo donde surge la duda

01

Abre el correo

En Gmail u Outlook, como siempre.

02

Pulsa «Escanear»

Se envía remitente, asunto, texto y enlaces. Nunca adjuntos.

03

Reglas + IA

Evidencias verificables combinadas con el modelo Jev.

04

Decide con evidencia

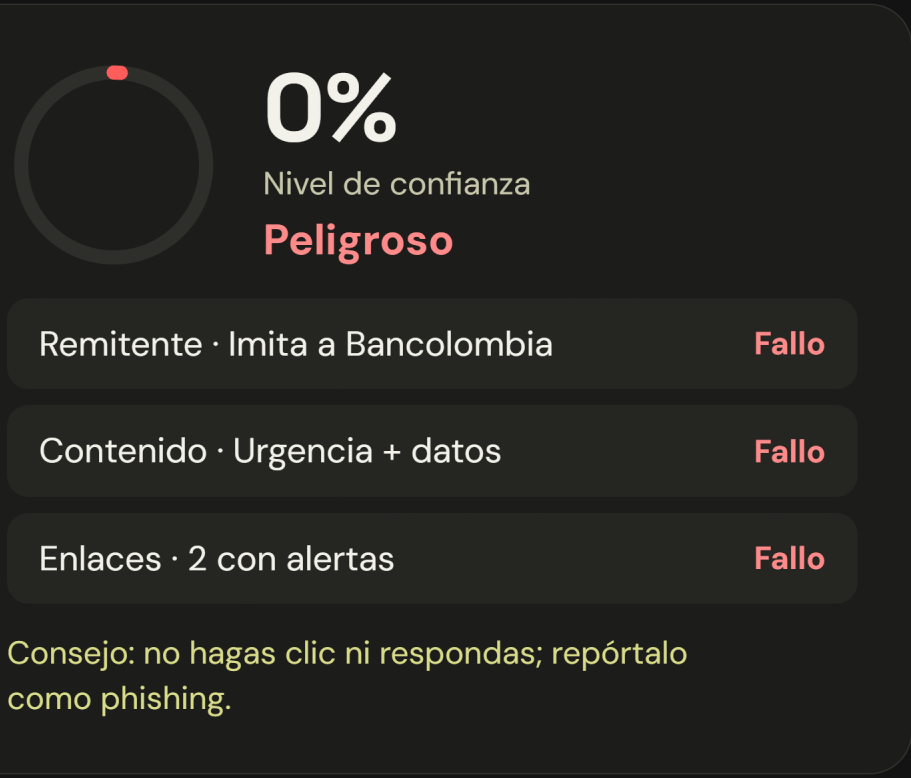
Puntaje, veredicto, señales y un consejo concreto.

UN ANÁLISIS REAL

Un veredicto que se explica solo

Correo de prueba: **seguridad@bancolombla-alertas.com** pide verificar la cuenta «en 24 horas» y enlaza a una dirección IP.

Cada señal dice qué se encontró, para que la persona aprenda a reconocerlo la próxima vez.



CASOS DE USO

Trece casos de uso, tres actores

Invitado

UC-01 Analizar correo abierto

- UC-02 Ver último resultado
- UC-03 Registrarse
- UC-04 Iniciar sesión
- UC-05 Recuperar contraseña

Usuario registrado

Todo lo del invitado, más:

- UC-06 Cerrar sesión
- UC-09 Consultar métricas
- UC-10 Ver detalle de un análisis
- UC-12 Editar perfil
- UC-13 Borrar historial

Sistema

Automático, sin intervención:

- UC-07 Guardar en historial
- UC-08 Sincronizar análisis offline
- UC-11 Controlar cuota diaria

ARQUITECTURA

Dos clientes, tres microservicios



Solo el gateway es público · los servicios internos exigen un token compartido · todo se valida con los mismos contratos Zod

¿POR QUÉ TRES?

Cada servicio es un contexto con su propia razón de cambiar

Criterio	Gateway	Análisis	Historial
Datos propios	Contadores de cuota	Caché de análisis	Análisis, perfiles, métricas
Cambia cuando…	cambian las reglas de acceso	cambia el modelo o las reglas	cambia el esquema de datos
Escala por…	tráfico total	costo y latencia de la IA	lecturas del panel
Si falla…	—	responden las reglas locales	el análisis queda en cola local

PATRONES DE DISEÑO

Un patrón por capa, no uno para todo

Microservicios

Hexagonal + casos de uso

El dominio no depende de Hono, de Supabase ni del proveedor de IA.

Extensión y panel web

MVVM

React es la Vista; los hooks **useScanViewModel** y **useAccountViewModel** son los ViewModels.

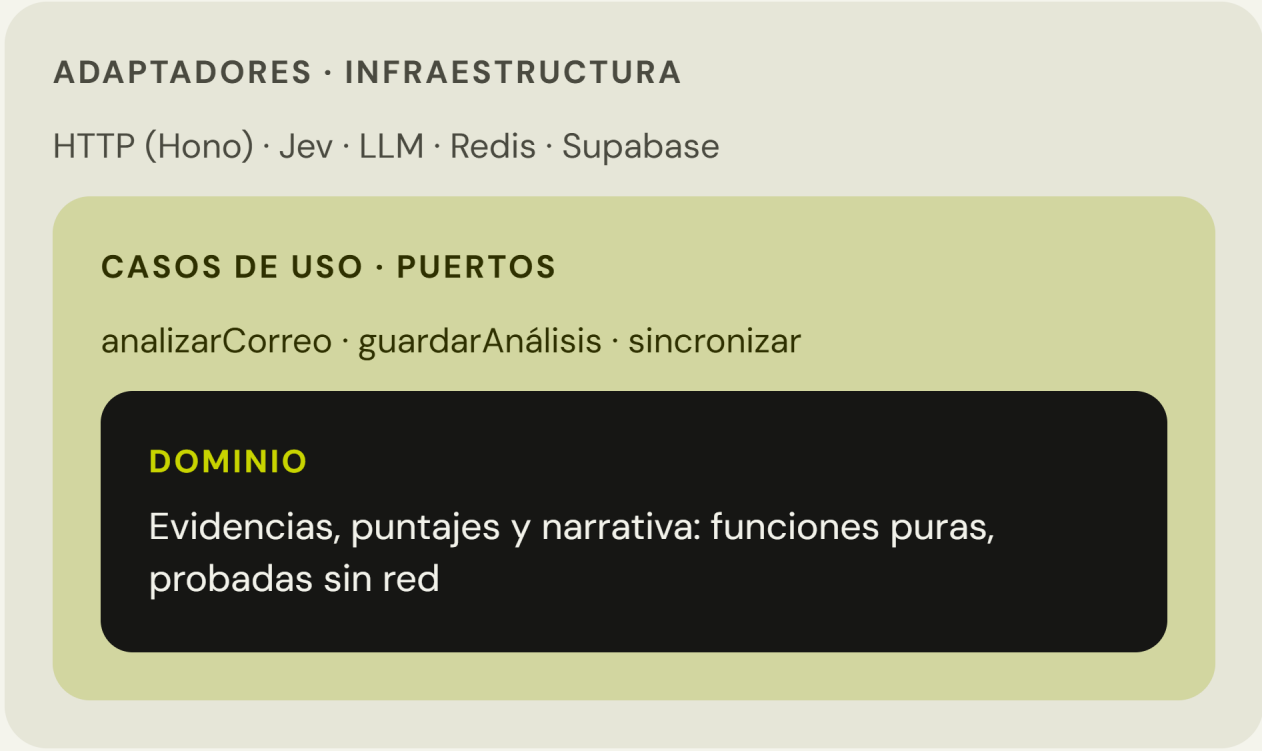
¿Por qué no MVC? En React la entrada llega directo a los componentes; un Controller separado sería código de relleno.

¿Por qué no MVP? El Presenter manda órdenes a la vista; React funciona con estado reactivo, que es lo que modela MVVM.

DENTRO DE UN SERVICIO

El dominio no depende de nada

Las dependencias apuntan hacia adentro. Cambiar Jev por otro modelo es escribir un adaptador, sin tocar las reglas.



MOTOR DE DECISIÓN

Detectar phishing es decidir, no conversar

	LLM de chat	Jev
Salida	Texto que hay que limpiar	Respuestas tipadas
Incertidumbre	No la expresa bien	Probabilidad por respuesta
Formato	Puede inventar campos	Opciones definidas por nosotros
Costo	Alto para respuestas cortas	Diseñado para decisiones rápidas

Una sola llamada responde:

verdict · elige seguro, sospechoso o peligroso

trust · puntaje en una rúbrica de 5 niveles

impersonation, pressure, sensitiveRequest · sí/no con probabilidad

link_0 … link_9 · riesgo de cada enlace

DEFENSA EN PROFUNDIDAD

La IA nunca decide sola

70/30

modelo / reglas verificables

Gana la señal más grave

Si el modelo dice «seguro» pero el dominio imita a un banco, la alerta se mantiene.

Si la IA falla, responden las reglas

El usuario recibe un resultado marcado como «respaldo» en vez de un error.

Caché por huella SHA-256

Repetir un análisis no gasta IA ni cuota.

SEGURIDAD Y PRIVACIDAD

Privacidad por diseño, no por promesa

Datos mínimos

Remitente, asunto, texto y enlaces. Nunca adjuntos.

Permisos mínimos

Solo la pestaña activa, cuando el usuario pulsa el botón.

Aislamiento por usuario

RLS en Postgres: nadie lee datos ajenos.

Servicios cerrados

Solo el gateway es público; el resto exige un token.

Validación en cada borde

Los mismos contratos Zod en clientes y servidores.

Nada inventado

No afirma SPF, DKIM ni DMARC sin ver los encabezados.

CALIDAD

Verificado, no solo programado

57
pruebas automáticas pasando

10
proyectos con TypeScript estricto

13
casos de uso documentados

1
comando para levantar todo: bun run dev

Las pruebas cubren las heurísticas, la combinación modelo + reglas, el adaptador de Jev, el respaldo ante fallos, la cuota, el historial y el panel. La migración SQL se probó en Postgres real, dos veces seguidas.

TECNOLOGÍAS

Un solo lenguaje de punta a punta: TypeScript

Capa	Tecnología
Extensión	React 19 · Vite · CRXJS (Chrome Manifest V3)
Panel web	Next.js 16 · Apache ECharts · Tailwind CSS
Microservicios	Bun · Hono
Inteligencia artificial	Jev (TypeSafe AI) · LLM compatible con OpenAI
Datos y autenticación	Supabase: Postgres con RLS + Auth
Contratos y monorepo	Zod · Turborepo · Bun workspaces · bun test

DESPLIEGUE

En línea con planes gratuitos

Render

Los 3 microservicios en Docker, creados con un Blueprint (render.yaml).

Vercel

El panel web en Next.js, plan Hobby.

Supabase

Autenticación y Postgres, plan Free.

En local basta **bun run dev**: sin claves, el análisis funciona con las reglas y en modo invitado.

PRÓXIMOS PASOS

Lo que sigue, cada uno como un adaptador nuevo

Autenticación del correo

Leer SPF, DKIM y DMARC desde «mostrar original».

Reputación de enlaces

Consultar Google Safe Browsing o VirusTotal.

Análisis automático

Revisar cada correo al abrirlo, si la persona lo activa.

Reporte de falsos positivos

Que el usuario corrija al sistema y mejore las reglas.

MAIL GUARD

Gracias

¿Preguntas? Puedo mostrar el análisis en vivo sobre un correo real.

github.com/bgrados31/uts-mail-guard · [correo de contacto]